

Relazione TPSIT – Problemi Classici della Programmazione Concorrente



Gruppo Iossa - Quattrini

Classe 5AM

Indice

- [Introduzione alla Programmazione Concorrente](#)
- [Problema Produttore–Consumatore](#)
- [Problema Lettori–Scrittori](#)
- [Problema dei Filosofi a Cena](#)
- [Problema del Barbiere Sonnolento](#)
- [Cosa sono i semafori in java](#)
- [Conclusioni](#)

Introduzione alla Programmazione Concorrente

In informatica, la programmazione concorrente è la tecnica che permette a più attività (thread) di essere eseguite “in parallelo” o comunque in modo interlacciato. ([SEZIONE CRITICA](#))

È molto utilizzata nei moderni sistemi multiprocessore e nei software che gestiscono più eventi contemporanei.

Il problema principale

Quando più thread accedono a risorse condivise (variabili, buffer, file, strutture dati), possono verificarsi:

- **Race condition:** due thread accedono contemporaneamente ai dati causando inconsistenze.
- **Deadlock:** due o più thread restano bloccati aspettando risorse che non si libereranno mai.
- **Starvation:** un thread non ottiene mai la CPU o una risorsa.
- **Interferenze:** un thread modifica dati mentre un altro li sta usando.

Per prevenire questi problemi si usano primitive di sincronizzazione come [Synchronized\(\)](#), [Wait\(\)](#), [Notify\(\)](#), semafori o monitor.

Cos'è una Sezione Critica

È una porzione di codice in cui processi o thread concorrenti accedono a risorse condivise, come dati comuni o file, che devono essere gestite con mutua esclusione per evitare incoerenze; ciò significa che solo un thread alla volta può eseguirla per garantire la consistenza dei dati, utilizzando meccanismi di sincronizzazione.

Perché è importante:

- **Evita la corruzione dei dati:** Senza controllo, più processi potrebbero modificare contemporaneamente la stessa variabile, portando a stati errati (es. due utenti che comprano lo stesso biglietto).
- **Garantisce la coerenza:** Assicura che le operazioni su dati condivisi avvengano in sequenza, preservando la logica del programma.

Synchronized(), Wait(), Notify()

Synchronized()

Il keyword `synchronized` in Java serve per proteggere l'accesso a un blocco di codice o a un metodo in modo che solo un thread alla volta possa eseguirlo per uno stesso oggetto.

Quando un thread entra in un metodo o blocco `synchronized`, acquisisce un "lock" (semaforo) sull'oggetto; gli altri thread devono aspettare finché quel lock non viene rilasciato.

Wait()

Il metodo `wait()` fa sì che il thread corrente si sospenda e rilasci il lock dell'oggetto, entrando nello stato di attesa finché un altro thread non chiama `notify()` o `notifyAll()` sullo stesso oggetto. Deve essere usato solo dentro un contesto `synchronized`, altrimenti viene lanciata un'eccezione..

Notify()

Il metodo `notify()` risveglia un singolo thread che è in attesa (`wait()`) sullo stesso oggetto. Anche questo deve essere chiamato dentro un blocco o metodo `synchronized`. Il thread svegliato non riprenderà l'esecuzione immediatamente: deve prima acquisire il lock che era stato rilasciato.

Problema Produttore–Consumatore

Il problema Produttore–Consumatore rappresenta una delle situazioni più tipiche nei sistemi concorrenti, in cui due entità distinte devono condividere un'area di memoria limitata, chiamata buffer.

Da una parte abbiamo il Produttore, che genera dati o elementi e si occupa di inserirli nel buffer; dall'altra il Consumatore, che preleva quegli stessi dati per elaborarli o utilizzarli.

Il buffer, essendo di dimensioni finite, introduce immediatamente due vincoli fondamentali:

- il produttore non può continuare a inserire elementi quando il buffer è pieno,
- il consumatore non può prelevare dati se il buffer è vuoto.

Queste limitazioni fanno sì che i due thread debbano coordinarsi, poiché l'uno dipende dallo stato lasciato dall'altro. In un contesto concorrente, l'assenza di sincronizzazione potrebbe portare a:

- race condition, ad esempio accessi simultanei che modificano gli stessi dati in modo non prevedibile;
- letture o scritture errate, come il consumatore che legge dati ancora non prodotti;
- inconsistenza dello stato del buffer, soprattutto quando più produttori o più consumatori agiscono insieme.

La soluzione classica prevede l'uso di meccanismi come [wait\(\)](#), [notify\(\)](#) e [synchronized\(\)](#), che permettono di garantire che:

- il produttore si sospenda automaticamente quando il buffer è pieno,
- il consumatore si sospenda quando il buffer è vuoto,
- e soprattutto che solo un thread alla volta possa accedere alla risorsa condivisa.

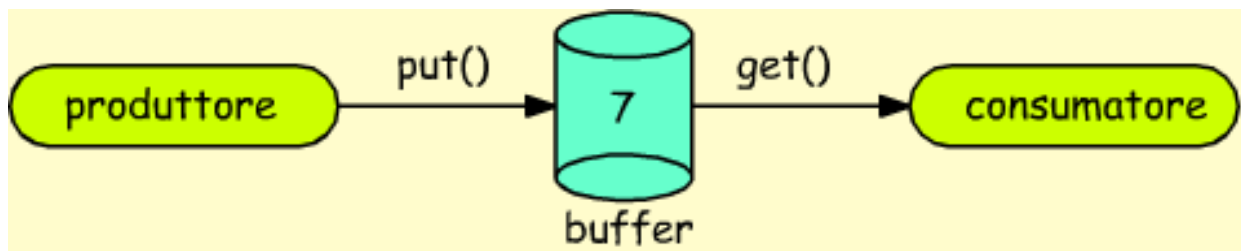
Codice Java

```
class Buffer {
    private int value;
    private boolean available = false;

    public synchronized void put(int v) throws InterruptedException
    {
        while (available) {
            wait();
        }
        value = v;
        available = true;
        System.out.println("Prodotto: " + v);
        notifyAll();
    }

    public synchronized int take() throws InterruptedException {
        while (!available) {
            wait();
        }
        available = false;
        System.out.println("Consumato: " + value);
        notifyAll();
        return value;
    }
}
```

Schema



Problema Lettori–Scrittori

Il problema Lettori–Scrittori riguarda l'accesso concorrente a una risorsa condivisa, come un file, un database o una struttura dati in memoria. La caratteristica principale di questo problema è la distinzione tra due categorie di thread:

- i Lettori, che necessitano soltanto di leggere la risorsa senza modificarla;
- gli Scrittori, che invece devono modificarla e quindi richiedono un controllo totale sulla risorsa durante l'operazione.

Più lettori possono agire contemporaneamente senza interferire tra loro. Tuttavia, quando uno scrittore intende operare, la situazione cambia completamente: per evitare inconsistenze, è necessario che:

- nessun lettore stia leggendo in quel momento,
- nessun altro scrittore sia già attivo.

In pratica gli scrittori devono avere accesso esclusivo, mentre i lettori possono condividere l'accesso.

Il problema porta a considerare alcune complicazioni, come:

- il rischio che un numero elevato di lettori impedisca agli scrittori di ottenere il controllo (*starvation degli scrittori*);
- la necessità di trovare un giusto equilibrio tra reattività alle letture e necessità di aggiornare i dati;
- l'impossibilità di permettere letture durante una scrittura, per evitare dati incoerenti.

Per affrontare queste problematiche si utilizzano costrutti come ReadWriteLock, che separano il concetto di “blocco in lettura” da quello di “blocco in scrittura”, consentendo così una gestione ottimizzata e sicura, i semafori o i monitor.

Codice Java

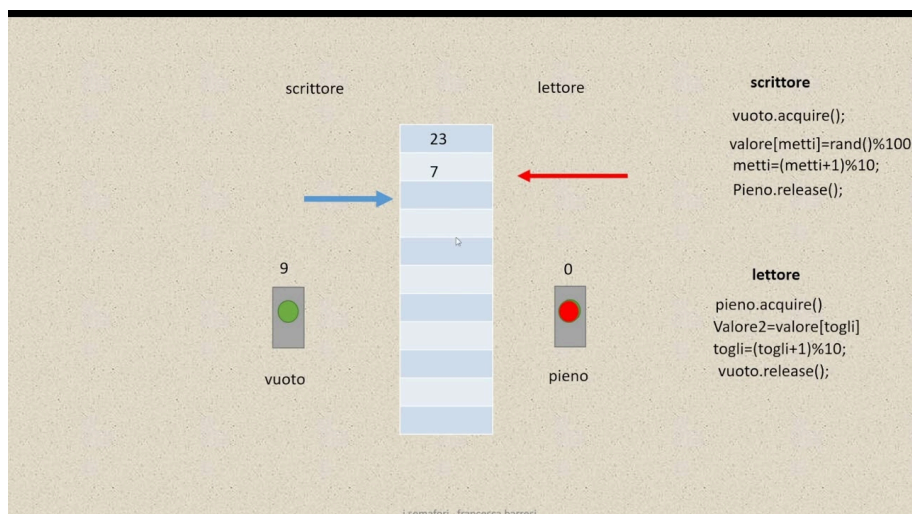
```
import java.util.concurrent.locks.*;

class Risorsa{
    private int data = 0;
    private final ReadWriteLock lock = new ReentrantReadWriteLock();

    public void Lettori() {
        lock.readLock().lock();
        try {
            System.out.println("Lettore legge: " + data);
        } finally {
            lock.readLock().unlock();
        }
    }

    public void Scrittore(int value) {
        lock.writeLock().lock();
        try {
            System.out.println("Scrittore scrive: " + value);
            data = value;
        } finally {
            lock.writeLock().unlock();
        }
    }
}
```

Schema



Problema dei Filosofi a Cena

Descrizione

Nel problema dei 5 filosofi, detto anche dei filosofi a cena, ci sono 5 filosofi che cenano intorno a un tavolo rotondo.

Ogni filosofo ha bisogno di due forchette per mangiare, ma ci sono solo 5 forchette tante quante i filosofi, ciascuna condivisa tra due filosofi.

Ogni filosofo opera secondo il seguente ciclo:

- 1. pensa per un po', poi gli viene fame;
- 2. cerca di procurarsi le forchette per mangiare;
- 3. dopo essere riuscito a prendere le due forchette, il filosofo mangia per un po' e poi lascia le forchette e ricomincia a pensare.

Questo problema fa riflettere due problematiche che sono:

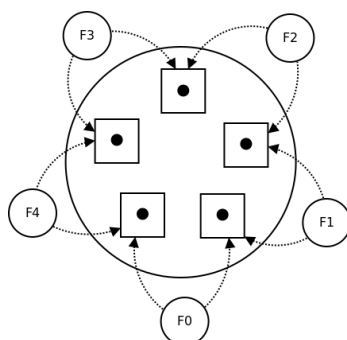
-Deadlock: si verifica se ciascuno dei filosofi ha in mano una sola forchetta e attende di prendere l'altra;

-Starvation: avviene se almeno uno dei filosofi non riesce mai a prendere entrambe le forchette.

Codice Java

```
class Forchetta {}  
class Filosofo extends Thread {  
    private Forchetta prima, seconda;  
    public Filosofo(Forchetta f1, Forchetta f2, boolean inverti) {  
        if (inverti) {  
            prima = f2;  
            seconda = f1;  
        } else {  
            prima = f1;  
            seconda = f2;  
        }  
    }  
    public void run() {  
        try {  
            while (true) {  
                pensa();  
                synchronized (prima) {  
                    synchronized (seconda) {  
                        mangia();  
                    }  
                }  
            }  
        } catch (InterruptedException e) {}  
    }  
    private void pensa() throws InterruptedException {  
        System.out.println(getName() + " pensa");  
        Thread.sleep(500);  
    }  
    private void mangia() throws InterruptedException {  
        System.out.println(getName() + " mangia");  
        Thread.sleep(500);  
    }  
}
```

Schema



Problema del Barbiere Sonnolento

Descrizione

Nel problema del barbiere sonnolento c'è un barbiere, una sola poltrona per far sedere il cliente per tagliarsi i capelli e per far riposare il barbiere, un numero definito di sedie per i clienti in attesa e i clienti che richiedono il servizio al barbiere.

Questo problema può essere suddiviso in tre parti:

-Nella prima parte, ovvero quando non ci sono i clienti, il barbiere rimane in attesa sulla poltrona.

-Nella seconda parte, ovvero quando entra un solo cliente, il cliente sveglia il barbiere e richiede il servizio al barbiere

-Nella terza parte, ovvero quando entrano molti clienti, se le sedie di attesa sono ancora disponibili il cliente si mette in attesa se invece le sedie di attesa sono tutte occupate il cliente se ne va

Codice Java

```
//SEMAFORI
import java.util.concurrent.Semaphore;

class Barberia {
    int sedieLibere;
    Semaphore clienti = new Semaphore(0);
    Semaphore barbiere = new Semaphore(0);
    Semaphore mutex = new Semaphore(1);

    public Barberia(int n) {
        sedieLibere = n;
    }

    public void entraCliente() throws InterruptedException {
        mutex.acquire();
        if (sedieLibere > 0) {
            sedieLibere--;
            System.out.println("Cliente si siede. Sedie libere: " +
sedieLibere);
            clienti.release();    // avvisa il barbiere
            mutex.release();
            barbiere.acquire();   // aspetta il barbiere
        } else {
            System.out.println("Cliente va via (nessuna sedia)");
            mutex.release();
        }
    }

    public void serviCliente() throws InterruptedException {
        clienti.acquire();        // dorme se non ci sono clienti
        mutex.acquire();
        sedieLibere++;
        System.out.println("Barbiere prende un cliente. Sedie
libere: " + sedieLibere);
        barbiere.release();      // chiama il cliente
        mutex.release();

        Thread.sleep(2000);      // tempo taglio
    }
}
```

```

class Barbieri extends Thread {
    Barberia barberia;

    public Barbieri(Barberia b) {
        barberia = b;
    }

    public void run() {
        try {
            while (true)
                barberia.serviCliente();
        } catch (InterruptedException e) {}
    }
}

```

```

class Cliente extends Thread {
    Barberia barberia;

    public Cliente(Barberia b) {
        barberia = b;
    }

    public void run() {
        try {
            barberia.entraCliente();
        } catch (InterruptedException e) {}
    }
}

```

Schema



Cosa sono i semafori in java

Un semaforo in Java è:

Un oggetto che ha il compito di far entrare solo un certo numero di thread in un "posto" alla volta, e di far aspettare gli altri in fila.

Conclusioni

Questi quattro problemi rappresentano la base per comprendere il funzionamento della programmazione concorrente.

Studiare e implementare questi esempi permette di imparare:

- come sincronizzare thread,
- come evitare condizioni di competizione,
- come prevenire deadlock e starvation,
- come gestire risorse condivise.

Sono modelli molto utilizzati in sistemi operativi, database, reti e applicazioni moderne.